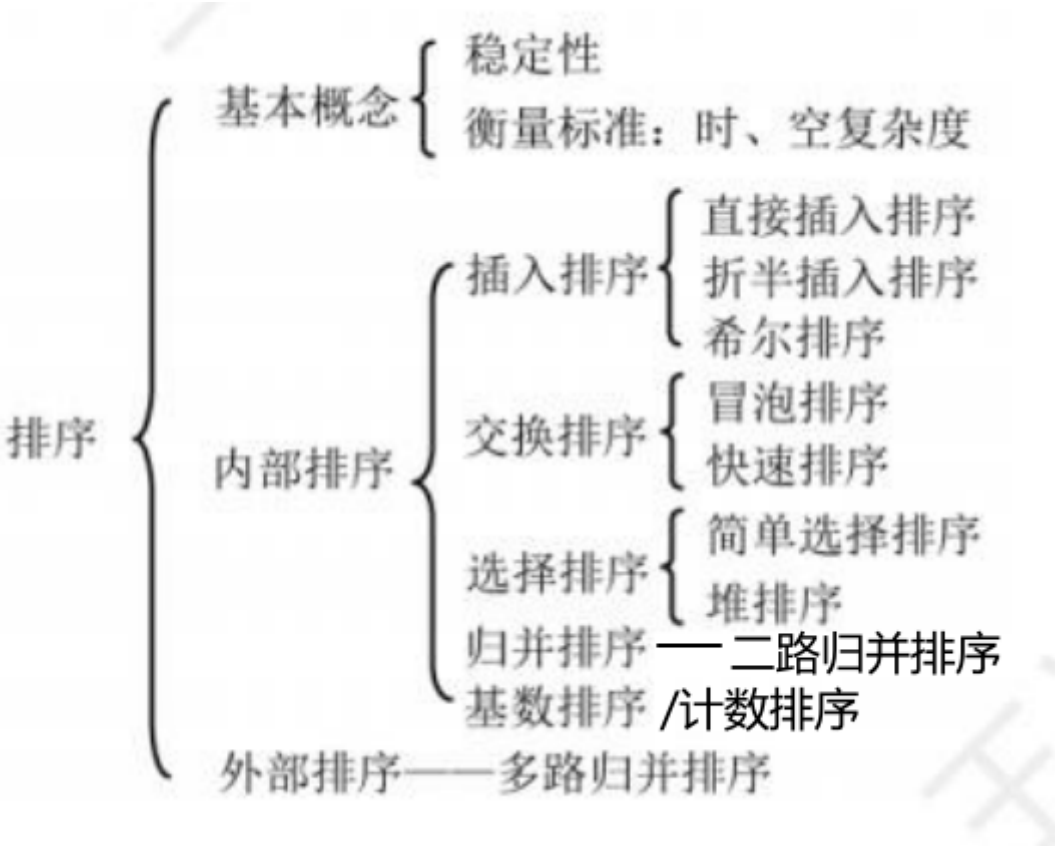
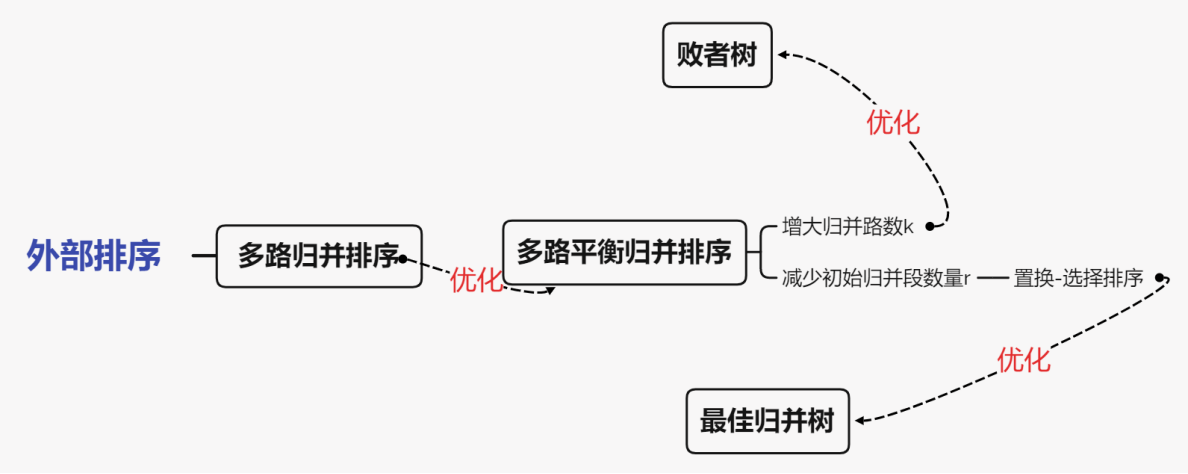
8.排序——注意：排序的一趟处理是指相应排序函数的一层调用，在快排中不仅仅对应1次，第一趟肯定是一次，但若是第一趟的基准元素不在序列首端或者尾端则第二趟(层)需要两次排序(左右两个子序列)

1.排序算法种类—内部排序/外部排序

内部排序五大类，后三大类（选择、归并、基数）不受初始序列的影响，时间数量级不变





2.内部排序算法

1.性质表(时、空、稳定)

表 8.1　各种排序算法的性质

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
折半插入排序					
√ 直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
√ 冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序	O(n^1.3)实则无法精确		$O(n^2)$	$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
√ 二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
√ 基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是
√ 计数排序	O(n)	O(n+k)	O(nlogn)	O(n+k)	是

2.最好最坏情况表

算法种类	最好情况	最坏情况
直接插入排序	初始有序	初始逆序
希尔排序	初始“正序”即基本有序，不确定/NULL	初始逆序
冒泡排序	初始有序	初始逆序
快速排序	每次分为2个长度相近子表(即两子表长度之差为1/0);	初始基本有序/基本逆序
简单选择排序	初始有序	初始无序
堆排序	NULL	NULL
二路归并排序		
基数排序		

3.关于各种算法的特点(最好最坏情况、对比次数、移动次数、时空、稳定性)的补充:

折半插入排序: 最好最坏指移动次数；**查找比较次数与初始状态无关**，只与表中元素个数有关，查找比较$n\log_2 n$，移动n^2
与直接插入排序最大不同是，查找次数固定，即每次选择新元素不与其前面最后一个有序元素相比较，而是直接二分查找他即将插入的位置，好处：统一了查找操作；坏处：即使元素位置不变也要查找，虽然最终查找到的也是他当前的位置

对元素个数相同的不同初始序列的进行排序时，总的**比较次数**(无移动)相同：严格来说**只有折半插入排序、简单选择排序**
而**堆排序只是在数量级上相同**，但具体比较次数可能还是有些许差异的。
　　简单选择明显不变没什么好说的，**折半插入排序为什么比较次数也是不变**：折半插入将**查找**与**插入**(即移动)两个操作分离开来，即每一趟必须进行查找操作，**即使有序也要查找。**
(是因为折半插入排序在选取插入的位置时必须查到一个长度为1的子序列才结束不会因为查到一个与其此次插入值相同或者相差不大的值的位置时而结束，因为他是跳着查询的不能确定周围是否还有与此值相同或者相近的其他值，这也是**为了保证稳定性**（举个例子：所有值相等的成初始序列），而每次又是二分划分的，所以相同长度子序列，**每轮对比次数都固定的**相等的所以总比较次数也必然相等)

简单选择排序：最好最坏指移动次数；**查找比较次数于初始状态无关**，只与表中元素个数有关，且移动次数很少，
移动≤3(n-1)，**查找比较n(n-1)/2——选择排序的比较次数与序列状态无关（简单选择、堆排）**
简单选择排序的不稳定性——来源自没有辅助数组，而是在原表上直接进行选择交换

冒泡排序：一定不能忘记**最终**遍历整个表来**确定**表内已经没有逆序了即释放已经有序/排序完成的信号

快排：最好与平均时空间复杂度为O($\log_2$n)，最坏时空间复杂度为O(n)；只有快排的辅助空间受初始序列状态影响1

堆排序：第一次建堆O(n)，之后的n-1次调整O($\log_2$n)；
建堆时：先按照顺序排成一个完全二叉树，从**最后一个分支结点**开始建堆调整交换
建堆过程：关键字总比较次数≤4n，即O(n)；
n-1次调整过程(即每次选出最大/最小根结点)：调整时间与树高有关，即O(h)=O($\log_2$n)
插入，删除：均为O($\log_2$n)，插入即放到堆末端然后调整，删除即将堆末端补根结点然后调整

基数排序：时间来源就只有2个——**分配+收集=n+r**

归并排序：归并排序是**外部排序的主要应用算法**，题目若是出现不一定只是指内部排序，若是涉及到很大的文件排序即内存一次性放不下有可能就是指外部排序的归并排序算法。

堆排序与希尔排序都利用到了随机访问的特性，所以要求必须用顺序存储，若是使用链式存储则效率会大大下降

2.对比（时、空、稳定、存储）

1.空间上：
　3个需要辅助空间：快排—递归工作栈；二路归并排序—辅助空间(用于元素复制)；基数排序—辅助链队列
　n很大时：**归并(n) > 快排($\log_2$n) > 基数(r) > 堆排(1)**

2.时间上：

3个平均最好O(nlog$_2$n)——快排、堆排、二路归并，快排实际应用中一般最优，但最坏时为O(n^2);

快排与二路归并都用了分治的思想，但二路归并其分割子序列与初始序列的排列无关，所以最好最坏都一样,快排不然。

最好情况下，时间复杂度**可以达到线性时间O(n)**的排序算法**只有2.5个**：**直接插入排序(折半不行)、冒泡排序**、计数也可算

2.1.时间复杂度无变化(即**Tn**不受初始序列状态影响)：也即最好最坏时时间复杂度都一样的，但情况可能不一样

简单选择、堆排、归并、基数的**T(n)**在所有情况数量级均不变

2.2.趟数不受初始序列状态影响的：（特殊存在—计数排序——1趟）

一定为**n-1趟**：**直接/折半插入、简单选择、堆排**（堆排的建堆过程不算入趟数里，但不要忘记这个操作的存在）

希尔排序——不确定；二路归并——(向上取整)log$_2$n；基数归并——数字的最大位数D，注意不是r

（**趟数受初始序列影响—冒泡、快排**）

（快排最好时log$_2$n趟，最坏时n-1趟，其余情况视初始序列而定）

3.稳定的算法：

4个——直冒二归基，可再延伸俩——折半与计数；稳定的O(nlog$_2$n)只有二路归并

4.适用的存储方式：

折半、希尔、快排、堆排(、计数)只适用于顺序存储；其余可顺可链。

5.过程特征：

冒泡、简单选择、堆排在每趟处理后都能产生当前的最大值或最小值来确定**一个**元素最终位置；

而快排一趟处理**至少**能直接确定**一个**元素的最终位置

3.实际应用中的对比选择

1）若 n 较小，可采用直接插入排序或简单选择排序。由于直接插入排序所需的记录移动次数较简单选择排序的多，因此当记录本身信息量较大时，用简单选择排序较好。

2）若 n 较大，应采用时间复杂度为 $O(n\log_2 n)$的排序算法：快速排序、堆排序或归并排序。当待排序的关键字随机分布时，快速排序被认为是目前基于比较的内部排序算法中最好的算法。堆排序所需的辅助空间少于快速排序，且不会出现快速排序可能的最坏情况，这两种排序都是不稳定的。若要求稳定且时间复杂度为 $O(n\log_2 n)$，可选用归并排序。

3）若文件的初始状态已按关键字基本有序，则选用直接插入或冒泡排序为宜。

4）在基于比较的排序算法中，每次比较两个关键字的大小之后，仅出现两种可能的转移，因此可以用一棵二叉树来描述比较判定过程，由此可以证明：当文件的 n 个关键字随机分布时，任何借助于“比较”的排序算法，至少需要 $O(n\log_2 n)$的时间。

5）若 n 很大，记录的关键字位数较少且可以分解时，采用基数排序较好。

6）当记录本身信息量较大时，为避免耗费大量时间移动记录，可用链表作为存储结构。

1）中为什么不用冒泡排序，首先n较小时可考虑这三种算法是因为n较小时它们简单且综合效率高；排除冒泡是因为冒泡排序在比较与移动方面都有不足，例如当基本有序时，他的比较次数大于直接插入排序，移动次数大于简单选择排序

3）中为什么不用简单选择排序，因为简单选择只与n有关，即使基本有序，每次仍然都是遍历n-i次（i=0,1，...n-2）

1中较好还是简单排序更优；3中还是直接插入更优，因为交换次数差不多的情况下，冒泡排序的对比次数更多

1．选取排序算法需要考虑的因素

1）待排序的元素个数 n。

2）待排序的元素的初始状态。

3）关键字的结构及其分布情况。

4）稳定性的要求。

5）存储结构及辅助空间的大小限制等。